DATABASE PROJECT

ROYAL FIELDS

PHASE 3

TI MAVUSO 40227375(GROUP LEADER), PD MATOOANE 31546455
S DLAMINI 39565769, A MAKWAKWA 35987782

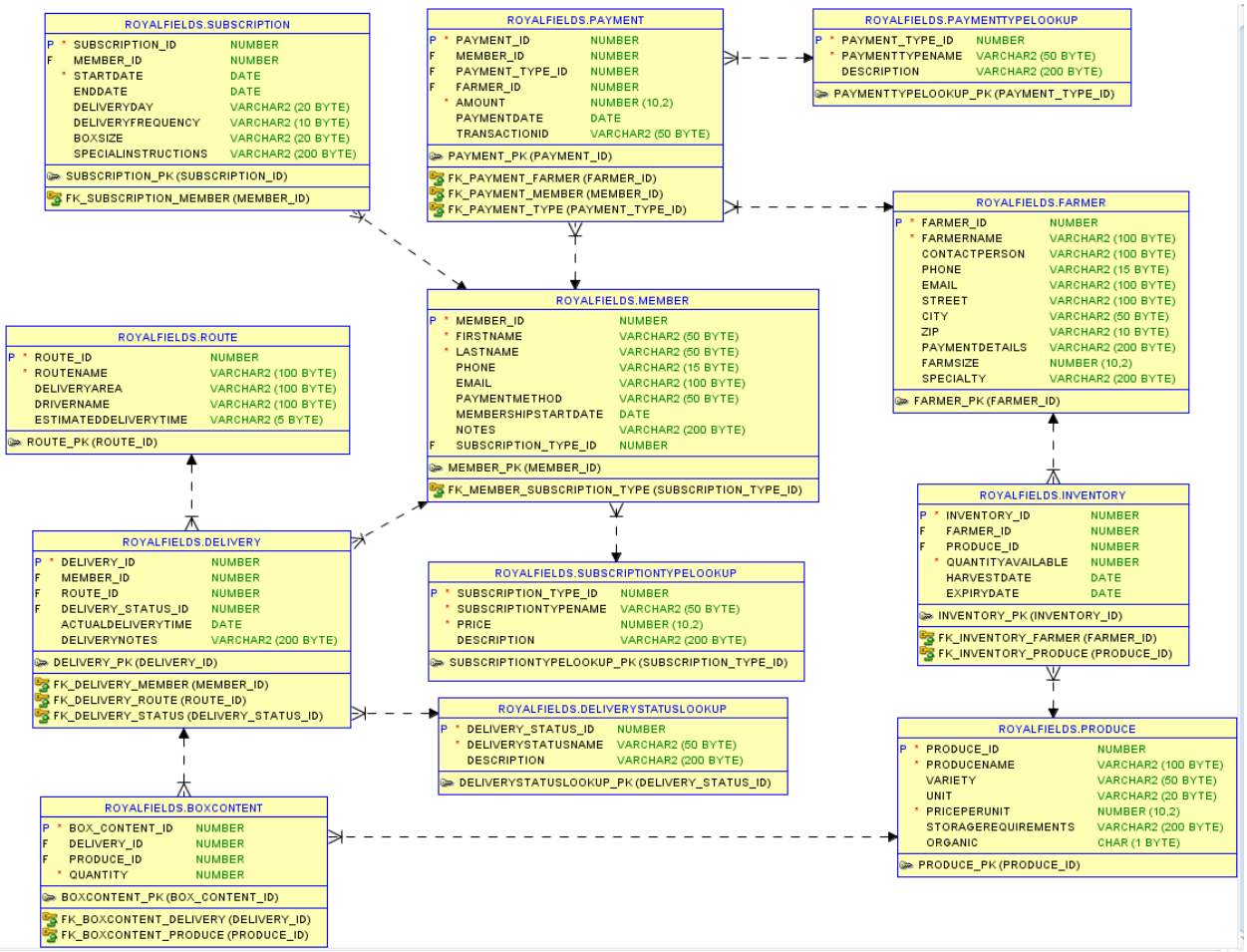
Table of Contents

1.Database Design/Layout....2
2.Database Objects....4
2.1. Removing All Tables to Allow the Creation of New Tables....4
2.2. Table Creation....4
2.2.1DELIVERYSTATUSLOOKUP Table....5
2.2.2. PRODUCE Table....5
2.2.3. FARMER Table....6
2.2.4. INVENTORY Table....7
2.2.5. PAYMENTTYPELOOKUP Table....8
2.2.6. SUBSCRIPTIONTYPELOOKUP Table....9
2.2.7. MEMBER Table....9
2.2.8. SUBSCRIPTION Table....10
2.2.9. ROUTE Table....11
2.2.10. DELIVERY Table....12
2.2.11. PAYMENT Table....12
2.2.12. BOXCONTENT Table....13
2.3. Populating the Database....14
2.4. Creating Views....15
2.4.1. Member Subscription View....15
2.4.2. Delivery Status View....16
2.5. Creating Indexes....16
2.5.1. SUBSCRIPTION Table Indexes....16
2.5.2. DELIVERY Table Indexes....17
2.5.3. INVENTORY Table Indexes....17
3.SQL Select Statements....18
3.1. Description of Usage....18
3.2. SQL SELECT Statements and Their Uses....19

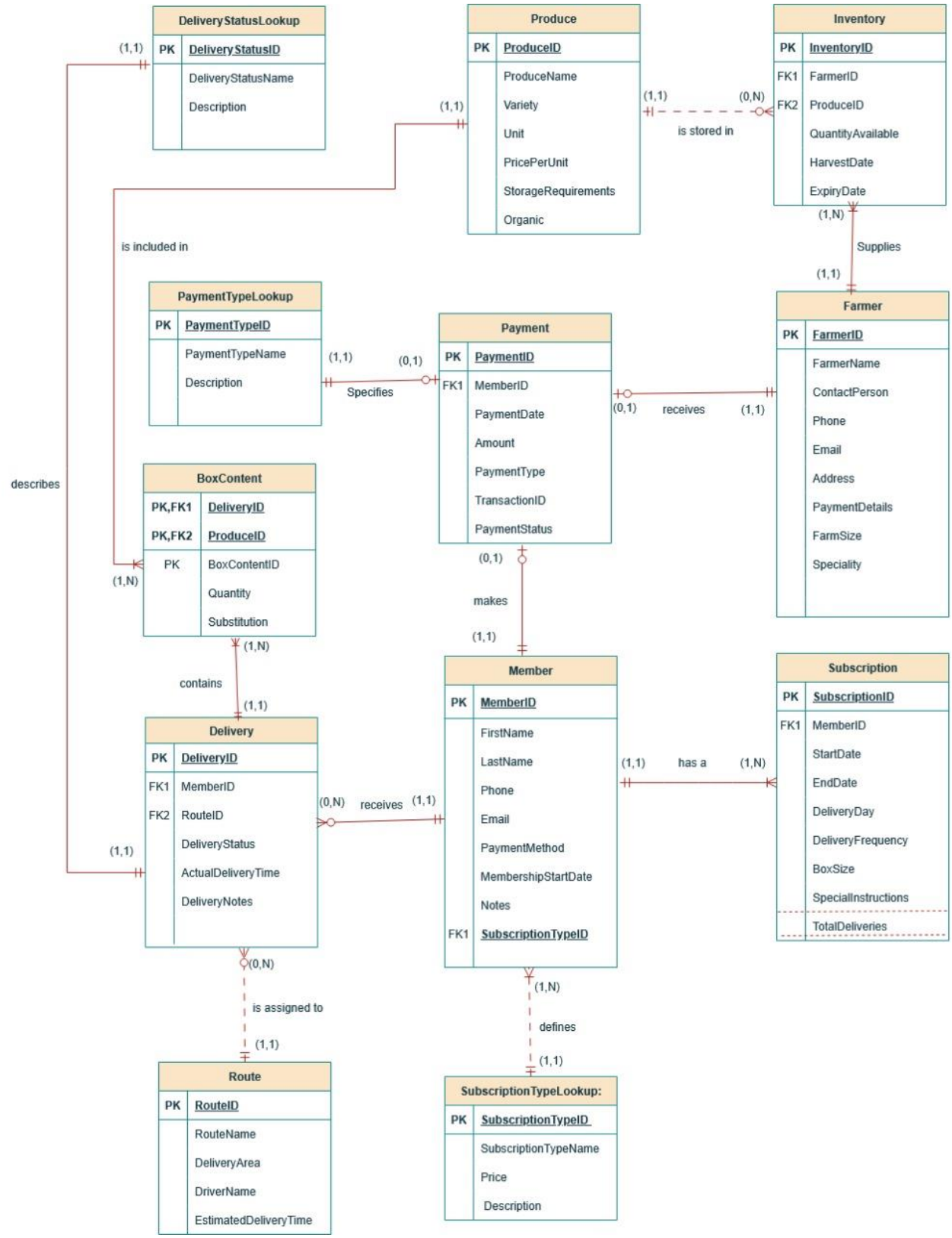
1.Database Design/Layout

This document presents the physical implementation of the Royal Fields Database System in Oracle SQL Developer, based on the Entity-Relationship Diagram (ERD) and logical model from Phase 2. The database supports Royal Fields, a community-supported agriculture (CSA) initiative in Limpopo, South Africa, connecting small-scale farmers with consumers. It manages subscriptions, produce inventory, deliveries, payments, and farmer relationships to address logistical challenges and promote sustainable agriculture. In the following sections, we create the database objects, populate them with data, and provide SQL statements to meet Royal Fields' information needs.

Physical Data Model in Oracle



Physical Data Model



2.Database Objects

This section details the creation of the database, including tables, relationships, indexes, views, and sample data. The database is designed in 3NF to ensure data integrity and efficiency, with primary and foreign keys to enforce relationships and check constraints to validate data.

2.1. Removing All Tables to Allow the Creation of New Tables

To prevent errors from duplicate tables, we drop existing tables and sequences in the correct order, starting with child entities to avoid foreign key conflicts. Dropping tables deletes all records, so this should be done cautiously in a production environment.

```
DROP TABLE BOXCONTENT;
DROP TABLE PAYMENT;
DROP TABLE DELIVERY;
DROP TABLE SUBSCRIPTION;
DROP TABLE MEMBER;
DROP TABLE ROUTE;
DROP TABLE INVENTORY;
DROP TABLE FARMER;
DROP TABLE PRODUCE;
DROP TABLE SUBSCRIPTIONTYPELOOKUP;
DROP TABLE PAYMENTTYPELOOKUP;
DROP TABLE DELIVERYSTATUSLOOKUP;

DROP SEQUENCE DeliveryStatusID_value;
DROP SEQUENCE ProduceID_value;
DROP SEQUENCE FarmerID_value;
DROP SEQUENCE InventoryID_value;
DROP SEQUENCE PaymentTypeID_value;
DROP SEQUENCE SubscriptionTypeID_value;
DROP SEQUENCE MemberID_value;
DROP SEQUENCE SubscriptionID_value;
DROP SEQUENCE RouteID_value;
DROP SEQUENCE DeliveryID_value;
DROP SEQUENCE PaymentID_value;
DROP SEQUENCE BoxContentID_value;
```

2.2. Table Creation

The following SQL statements create the 12 tables, assigning primary keys, foreign keys, and appropriate datatypes. We use NUMBER for IDs and numeric fields, VARCHAR2 for text, DATE for dates, and NUMBER(1) for boolean-like fields (e.g., Organic). Check constraints ensure data validity, and sequences generate unique primary keys to maintain data integrity.

2.2.1 **DELIVERYSTATUSLOOKUP Table**

This table stores delivery status codes (e.g., Scheduled, Delivered) used in the DELIVERY table. It has no foreign keys and uses standard datatypes.

```
CREATE TABLE DELIVERYSTATUSLOOKUP (
    DELIVERY_STATUS_ID NUMBER(3) NOT NULL,
    DeliveryStatusName VARCHAR2(50) NOT NULL,
    Description VARCHAR2(200),
    CONSTRAINT DeliveryStatusID_PK PRIMARY KEY (DELIVERY_STATUS_ID)
);
```

The DeliveryStatusID_value sequence is created alongside this table to determine the value of the DELIVERY_STATUS_ID primary key. By using sequences, we minimize the risk of users entering invalid values into the primary key slot, ensuring data integrity and guaranteeing that each record has a unique primary key. The CYCLE keyword means that when the maximum value (100) is reached, the sequence will restart from the minimum value (1). If this occurs, there is a potential risk of duplicate primary keys, which can be addressed if the DELIVERY_STATUS_ID approaches the maximum value. Sequences are used across many tables in this database, so this explanation will not be repeated in detail for each table.

```
CREATE SEQUENCE DeliveryStatusID_value
    START WITH 1
    INCREMENT BY 1
    MINVALUE 1
    MAXVALUE 100
    CYCLE;
```

2.2.2. PRODUCE Table

The PRODUCE table stores details of agricultural products available through Royal Fields, such as Amaranth (Morogo), Okra, and Mango, which are culturally relevant to Limpopo. It tracks pricing, storage requirements, and organic status to support inventory management and delivery planning. The Organic field uses NUMBER(1) (0 = No, 1 = Yes) as Oracle lacks a boolean datatype.

```
CREATE TABLE PRODUCE (
    PRODUCE_ID NUMBER(4) NOT NULL,
    ProduceName VARCHAR2(100) NOT NULL,
    Variety VARCHAR2(50),
    Unit VARCHAR2(20),
    PricePerUnit NUMBER(10,2) NOT NULL,
    StorageRequirements VARCHAR2(200),
    Organic NUMBER(1) CHECK (Organic IN (0,1)) NOT NULL,
    CONSTRAINT ProduceID_PK PRIMARY KEY (PRODUCE_ID)
);
```

The ProduceID_value sequence is created to generate unique PRODUCE_ID values. Sequences prevent invalid primary key entries, ensuring data integrity and unique identification for each produce record. The CYCLE keyword allows the sequence to restart from the minimum value (1) after reaching the maximum (1000), which could lead to duplicate keys if the maximum is reached, an issue to monitor and address if necessary. This sequence approach is standard across tables.

```
CREATE SEQUENCE ProduceID_value
    START WITH 1
    INCREMENT BY 1
    MINVALUE 1
    MAXVALUE 1000
    CYCLE;
```

2.2.3. FARMER Table

The FARMER table stores information about small-scale farmers participating in Royal Fields, including contact details, farm size, and specialties (e.g., traditional vegetables). It supports relationships with the INVENTORY and PAYMENT tables to track produce supply and farmer payments. The Email and Phone fields are unique to prevent duplicates.

```
CREATE TABLE FARMER (
    FARMER_ID NUMBER(4) NOT NULL,
    FarmerName VARCHAR2(100) NOT NULL,
    ContactPerson VARCHAR2(100),
    Phone VARCHAR2(15) NOT NULL UNIQUE,
    Email VARCHAR2(100) UNIQUE,
    Street VARCHAR2(100),
    City VARCHAR2(50),
    Zip VARCHAR2(10),
    PaymentDetails VARCHAR2(200),
    FarmSize NUMBER(10,2),
    Specialty VARCHAR2(200),
    CONSTRAINT FarmerID_PK PRIMARY KEY (FARMER_ID)
);
```

The FarmerID_value sequence ensures unique FARMER_ID values, minimizing the risk of invalid primary key entries and maintaining data integrity. The CYCLE keyword restarts the sequence at the minimum value (1) after the maximum (1000), potentially causing duplicate keys if the maximum is reached, which should be addressed if the FARMER_ID nears the limit. This sequence explanation applies to other tables.

```
CREATE SEQUENCE FarmerID_value
    START WITH 1
    INCREMENT BY 1
    MINVALUE 1
    MAXVALUE 1000
    CYCLE;
```

2.2.4. INVENTORY Table

The INVENTORY table tracks the stock of produce supplied by farmers, linking to the FARMER and PRODUCE tables via foreign keys. It records quantities, harvest dates, and expiration dates to ensure Royal Fields can manage available produce for deliveries. A check constraint ensures non-negative quantities.

```
CREATE TABLE INVENTORY (
    INVENTORY_ID NUMBER(5) NOT NULL,
    FARMER_ID NUMBER(4) NOT NULL,
    PRODUCE_ID NUMBER(4) NOT NULL,
    QuantityAvailable NUMBER(10) NOT NULL CHECK (QuantityAvailable >= 0),
    HarvestDate DATE,
    ExpiryDate DATE,
    CONSTRAINT InventoryID_PK PRIMARY KEY (INVENTORY_ID),
    CONSTRAINT fk_inventory_farmer FOREIGN KEY (FARMER_ID) REFERENCES FARMER(FARMER_ID),
    CONSTRAINT fk_inventory_produce FOREIGN KEY (PRODUCE_ID) REFERENCES PRODUCE(PRODUCE_ID)
);
```

The InventoryID_value sequence generates unique INVENTORY_ID values, preventing invalid primary key entries and ensuring data integrity for each inventory record. The CYCLE keyword restarts the sequence at the minimum value (1) after the maximum (10000), which may cause duplicate keys if the maximum is reached, a concern to address if necessary. This sequence approach is used across tables.

```
CREATE SEQUENCE InventoryID_value
    START WITH 1
    INCREMENT BY 1
    MINVALUE 1
    MAXVALUE 10000
    CYCLE;
```

2.2.5. PAYMENTTYPELOOKUP Table

The PAYMENTTYPELOOKUP table stores standardized payment method codes (e.g., Credit Card, Bank Transfer, Cash) to ensure consistent payment types in the PAYMENT table. It supports financial transaction tracking in Royal Fields and has no foreign keys.

```
CREATE TABLE PAYMENTTYPELOOKUP (
    PAYMENT_TYPE_ID NUMBER(3) NOT NULL,
    PaymentTypeName VARCHAR2(50) NOT NULL,
    Description VARCHAR2(200),
    CONSTRAINT PaymentTypeID_PK PRIMARY KEY (PAYMENT_TYPE_ID)
);
```

The PaymentTypeID_value sequence ensures unique PAYMENT_TYPE_ID values, maintaining data integrity by preventing invalid primary key entries. The CYCLE keyword

allows the sequence to restart from the minimum value (1) after the maximum (100), potentially causing duplicate keys if the maximum is reached, which should be monitored. This applies to other tables' sequences.

```
CREATE SEQUENCE PaymentTypeID_value
    START WITH 1
    INCREMENT BY 1
    MINVALUE 1
    MAXVALUE 100
    CYCLE;
```

2.2.6. SUBSCRIPTIONTYPELOOKUP Table

The SUBSCRIPTIONTYPELOOKUP table defines subscription plans offered by Royal Fields (e.g., Standard, Premium), including pricing and descriptions. It standardizes subscription types for the MEMBER table, with a check constraint ensuring non-negative prices.

```
CREATE TABLE SUBSCRIPTIONTYPELOOKUP (
    SUBSCRIPTION_TYPE_ID NUMBER(3) NOT NULL,
    SubscriptionTypeName VARCHAR2(50) NOT NULL,
    Price NUMBER(10,2) NOT NULL CHECK (Price >= 0),
    Description VARCHAR2(200),
    CONSTRAINT SubscriptionTypeID_PK PRIMARY KEY (SUBSCRIPTION_TYPE_ID)
);
```

The SubscriptionTypeID_value sequence generates unique SUBSCRIPTION_TYPE_ID values, ensuring data integrity and preventing invalid primary key entries. The CYCLE keyword restarts the sequence at the minimum value (1) after the maximum (100), which could lead to duplicate keys if the maximum is reached, an issue to address if needed. This sequence approach is standard across tables.

```
CREATE SEQUENCE SubscriptionTypeID_value
    START WITH 1
    INCREMENT BY 1
    MINVALUE 1
    MAXVALUE 100
    CYCLE;
```

2.2.7. MEMBER Table

The MEMBER table stores details of Royal Fields customers (members), including contact information, payment methods, and subscription plans. It links to the SUBSCRIPTIONTYPELOOKUP table via a foreign key, with unique Email and Phone fields to prevent duplicates.

```
CREATE TABLE MEMBER (
    MEMBER_ID NUMBER(5) NOT NULL,
    FirstName VARCHAR2(50) NOT NULL,
    LastName VARCHAR2(50) NOT NULL,
    Phone VARCHAR2(15) NOT NULL UNIQUE,
    Email VARCHAR2(100) UNIQUE,
    PaymentMethod VARCHAR2(50),
    MembershipStartDate DATE,
    Notes VARCHAR2(200),
    SUBSCRIPTION_TYPE_ID NUMBER(3) NOT NULL,
    CONSTRAINT MemberID_PK PRIMARY KEY (MEMBER_ID),
    CONSTRAINT fk_member_subscription_type FOREIGN KEY (SUBSCRIPTION_TYPE_ID) REFERENCES SUBSCRIPTIONTYPELOOKUP(SUBSCRIPTION_TYPE_ID)
);
```

The MemberID_value sequence ensures unique MEMBER_ID values, preventing invalid primary key entries and maintaining data integrity. The CYCLE keyword allows the sequence to restart from the minimum value (1) after the maximum (10000), potentially causing duplicate keys if the maximum is reached, which should be monitored. This sequence explanation applies to other tables.

```
CREATE SEQUENCE MemberID_value
    START WITH 1
    INCREMENT BY 1
    MINVALUE 1
    MAXVALUE 10000
    CYCLE;
```

2.2.8. SUBSCRIPTION Table

The SUBSCRIPTION table tracks active subscriptions for Royal Fields members, including start/end dates, delivery schedules, and preferences (e.g., box size, special instructions). It links to the MEMBER table via a foreign key, with a check constraint limiting DeliveryFrequency to 'Weekly' or 'Bi-Weekly'.

```
CREATE TABLE SUBSCRIPTION (
    SUBSCRIPTION_ID NUMBER(5) NOT NULL,
    MEMBER_ID NUMBER(5) NOT NULL,
    StartDate DATE NOT NULL,
    EndDate DATE,
    DeliveryDay VARCHAR2(20),
    DeliveryFrequency VARCHAR2(10) CHECK (DeliveryFrequency IN ('Weekly', 'Bi-Weekly')) NOT NULL,
    BoxSize VARCHAR2(20),
    SpecialInstructions VARCHAR2(200),
    CONSTRAINT SubscriptionID_PK PRIMARY KEY (SUBSCRIPTION_ID),
    CONSTRAINT fk_subscription_member FOREIGN KEY (MEMBER_ID) REFERENCES MEMBER(MEMBER_ID)
);
```

The SubscriptionID_value sequence generates unique SUBSCRIPTION_ID values, ensuring data integrity by preventing invalid primary key entries. The CYCLE keyword restarts the

sequence at the minimum value (1) after the maximum (10000), which may cause duplicate keys if the maximum is reached, a concern to address if necessary. This sequence approach is used across tables.

```
CREATE SEQUENCE SubscriptionID_value
    START WITH 1
    INCREMENT BY 1
    MINVALUE 1
    MAXVALUE 10000
    CYCLE;
```

2.2.9. ROUTE Table

The ROUTE table defines delivery routes in Limpopo (e.g., Polokwane North, Tzaneen Central) for Royal Fields, including delivery areas, driver names, and estimated delivery times. It supports delivery logistics and has no foreign keys.

```
CREATE TABLE ROUTE (
    ROUTE_ID NUMBER(4) NOT NULL,
    RouteName VARCHAR2(100) NOT NULL,
    DeliveryArea VARCHAR2(100),
    DriverName VARCHAR2(100),
    EstimatedDeliveryTime VARCHAR2(5),
    CONSTRAINT RouteID_PK PRIMARY KEY (ROUTE_ID)
);
```

The RouteID_value sequence ensures unique ROUTE_ID values, preventing invalid primary key entries and maintaining data integrity. The CYCLE keyword allows the sequence to restart from the minimum value (1) after the maximum (1000), potentially causing duplicate keys if the maximum is reached, which should be monitored. This applies to other tables' sequences.

```
CREATE SEQUENCE RouteID_value
    START WITH 1
    INCREMENT BY 1
    MINVALUE 1
    MAXVALUE 1000
    CYCLE;
```

2.2.10. DELIVERY Table

The DELIVERY table tracks individual deliveries to Royal Fields members, including delivery dates, routes, and statuses. It links to the MEMBER, ROUTE, and DELIVERYSTATUSLOOKUP tables via foreign keys to manage delivery schedules and tracking.

```
CREATE TABLE DELIVERY (
    DELIVERY_ID NUMBER(5) NOT NULL,
    MEMBER_ID NUMBER(5) NOT NULL,
    ROUTE_ID NUMBER(4) NOT NULL,
    DELIVERY_STATUS_ID NUMBER(3) NOT NULL,
    ActualDeliveryTime DATE,
    DeliveryNotes VARCHAR2(200),
    CONSTRAINT DeliveryID_PK PRIMARY KEY (DELIVERY_ID),
    CONSTRAINT fk_delivery_member FOREIGN KEY (MEMBER_ID) REFERENCES MEMBER(MEMBER_ID),
    CONSTRAINT fk_delivery_route FOREIGN KEY (ROUTE_ID) REFERENCES ROUTE(ROUTE_ID),
    CONSTRAINT fk_delivery_status FOREIGN KEY (DELIVERY_STATUS_ID) REFERENCES DELIVERYSTATUSLOOKUP(DELIVERY_STATUS_ID)
);
```

The DeliveryID_value sequence generates unique DELIVERY_ID values, ensuring data integrity and preventing invalid primary key entries. The CYCLE keyword restarts the sequence at the minimum value (1) after the maximum (10000), which could lead to duplicate keys if the maximum is reached, an issue to address if needed. This sequence explanation applies to other tables.

```
CREATE SEQUENCE DeliveryID_value
    START WITH 1
    INCREMENT BY 1
    MINVALUE 1
    MAXVALUE 10000
    CYCLE;
```

2.2.11. PAYMENT Table

The PAYMENT table records financial transactions, including payments from members and to farmers, linking to the MEMBER, PAYMENTTYPELOOKUP, and FARMER tables. The MEMBER_ID and FARMER_ID foreign keys are nullable to allow flexibility (e.g., payments to farmers without a member). A check constraint ensures non-negative amounts.

```
CREATE TABLE PAYMENT (
    PAYMENT_ID NUMBER(5) NOT NULL,
    MEMBER_ID NUMBER(5),
    PAYMENT_TYPE_ID NUMBER(3) NOT NULL,
    FARMER_ID NUMBER(4),
    Amount NUMBER(10,2) NOT NULL CHECK (Amount >= 0),
    PaymentDate DATE,
    TransactionID VARCHAR2(50),
    CONSTRAINT PaymentID_PK PRIMARY KEY (PAYMENT_ID),
    CONSTRAINT fk_payment_member FOREIGN KEY (MEMBER_ID) REFERENCES MEMBER(MEMBER_ID),
    CONSTRAINT fk_payment_type FOREIGN KEY (PAYMENT_TYPE_ID) REFERENCES PAYMENTTYPELOOKUP(PAYMENT_TYPE_ID),
    CONSTRAINT fk_payment_farmer FOREIGN KEY (FARMER_ID) REFERENCES FARMER(FARMER_ID)
);
```

The PaymentID_value sequence ensures unique PAYMENT_ID values, preventing invalid primary key entries and maintaining data integrity. The CYCLE keyword allows the sequence to restart from the minimum value (1) after the maximum (10000), potentially causing duplicate keys if the maximum is reached, which should be monitored. This sequence approach is used across tables.

```
CREATE SEQUENCE PaymentID_value
    START WITH 1
    INCREMENT BY 1
    MINVALUE 1
    MAXVALUE 10000
    CYCLE;
```

2.2.12. BOXCONTENT Table

The BOXCONTENT table is a bridge table that resolves the many-to-many relationship between the DELIVERY and PRODUCE tables. It links each delivery to specific produce items and their quantities, enabling Royal Fields to track the contents of each delivery box accurately. For example, a single delivery can include multiple produce items (e.g., Amaranth and Okra), and a produce item can appear in multiple deliveries. A check constraint ensures positive quantities.

```
CREATE TABLE BOXCONTENT (
    BOX_CONTENT_ID NUMBER(5) NOT NULL,
    DELIVERY_ID NUMBER(5) NOT NULL,
    PRODUCE_ID NUMBER(4) NOT NULL,
    Quantity NUMBER(10) NOT NULL CHECK (Quantity > 0),
    CONSTRAINT BoxContentID_PK PRIMARY KEY (BOX_CONTENT_ID),
    CONSTRAINT fk_boxcontent_delivery FOREIGN KEY (DELIVERY_ID) REFERENCES DELIVERY(DELIVERY_ID),
    CONSTRAINT fk_boxcontent_produce FOREIGN KEY (PRODUCE_ID) REFERENCES PRODUCE(PRODUCE_ID)
);
```

The BoxContentID_value sequence generates unique BOX_CONTENT_ID values for each box content record, ensuring data integrity by preventing invalid primary key entries. The

CYCLE keyword restarts the sequence at the minimum value (1) after the maximum (10000), which could lead to duplicate keys if the maximum is reached, a concern to address if necessary. This sequence explanation applies to other tables.

```
CREATE SEQUENCE BoxContentID_value
    START WITH 1
    INCREMENT BY 1
    MINVALUE 1
    MAXVALUE 10000
    CYCLE;
```

2.3. Populating the Database

The following INSERT statements populate the tables with sample data to test queries and demonstrate the database's functionality. The data reflects the Royal Fields CSA context, including culturally relevant produce (e.g., Amaranth/Morogo), realistic farmer and member details, and subscription/delivery schedules (e.g., weekly or bi-weekly). Sequences ensure unique IDs, and foreign key constraints are respected to maintain data integrity.

DELIVERYSTATUSLOOKUP

```
INSERT INTO DELIVERYSTATUSLOOKUP VALUES (DeliveryStatusID_value.nextval, 'Scheduled', 'Delivery is planned');
INSERT INTO DELIVERYSTATUSLOOKUP VALUES (DeliveryStatusID_value.nextval, 'In Transit', 'Delivery is on the way');
INSERT INTO DELIVERYSTATUSLOOKUP VALUES (DeliveryStatusID_value.nextval, 'Delivered', 'Delivery completed');
```

PRODUCE

```
INSERT INTO PRODUCE VALUES (ProduceID_value.nextval, 'Amaranth', 'Morogo', 'kg', 15.00, 'Cool, dry place', 1);
INSERT INTO PRODUCE VALUES (ProduceID_value.nextval, 'Okra', 'Local', 'kg', 20.00, 'Refrigerated', 0);
INSERT INTO PRODUCE VALUES (ProduceID_value.nextval, 'Mango', 'Tommy Atkins', 'kg', 25.00, 'Room temperature', 1);
```

FARMER

```
INSERT INTO FARMER VALUES (FarmerID_value.nextval, 'Mopani Farms', 'John Mopani', '0821234567', 'john@mopanifarms.co.za', '12 Farm Rd', 'Polokwane', '0699', 'Bank Transfer', 10.5, 'Traditional vegetables');
INSERT INTO FARMER VALUES (FarmerID_value.nextval, 'Limpopo Greens', 'Sarah Tshabalala', '0839876543', 'sarah@limpopogreens.co.za', '45 Green St', 'Tzaneen', '0850', 'EFT', 8.0, 'Fruits');
```

INVENTORY

```
INSERT INTO INVENTORY VALUES (InventoryID_value.nextval, 1, 1, 50, TO_DATE('2025-04-20', 'YYYY-MM-DD'), TO_DATE('2025-05-20', 'YYYY-MM-DD'));
INSERT INTO INVENTORY VALUES (InventoryID_value.nextval, 1, 2, 30, TO_DATE('2025-04-22', 'YYYY-MM-DD'), TO_DATE('2025-05-15', 'YYYY-MM-DD'));
INSERT INTO INVENTORY VALUES (InventoryID_value.nextval, 2, 3, 100, TO_DATE('2025-04-18', 'YYYY-MM-DD'), TO_DATE('2025-05-10', 'YYYY-MM-DD'));
```

PAYMENTTYPELOOKUP

```
INSERT INTO PAYMENTTYPELOOKUP VALUES (PaymentTypeID_value.nextval, 'Credit Card', 'Visa/Mastercard');
INSERT INTO PAYMENTTYPELOOKUP VALUES (PaymentTypeID_value.nextval, 'Bank Transfer', 'EFT payment');
INSERT INTO PAYMENTTYPELOOKUP VALUES (PaymentTypeID_value.nextval, 'Cash', 'Cash on delivery');
```

SUBSCRIPTIONTYPELOOKUP

```
INSERT INTO SUBSCRIPTIONTYPELOOKUP VALUES (SubscriptionTypeID_value.nextval, 'Standard', 200.00, 'Weekly small box');
INSERT INTO SUBSCRIPTIONTYPELOOKUP VALUES (SubscriptionTypeID_value.nextval, 'Premium', 350.00, 'Bi-weekly large box');
```

MEMBER

```
INSERT INTO MEMBER VALUES (MemberID_value.nextval, 'Thandi', 'Ndlovu', '0812345678', 'thandi@gmail.com', 'Credit Card', TO_DATE('2025-04-01', 'YYYY-MM-DD'), 'Prefers traditional veggies', 1);
INSERT INTO MEMBER VALUES (MemberID_value.nextval, 'Peter', 'Mkhize', '0823456789', 'peter.mkhize@yahoo.com', 'Cash', TO_DATE('2025-04-10', 'YYYY-MM-DD'), NULL, 2);
```

SUBSCRIPTION

```
INSERT INTO SUBSCRIPTION VALUES (SubscriptionID_value.nextval, 1, TO_DATE('2025-04-01', 'YYYY-MM-DD'), TO_DATE('2025-12-31', 'YYYY-MM-DD'), 'Monday', 'Weekly', 'Small', 'Include morogo');
INSERT INTO SUBSCRIPTION VALUES (SubscriptionID_value.nextval, 2, TO_DATE('2025-04-10', 'YYYY-MM-DD'), TO_DATE('2025-12-31', 'YYYY-MM-DD'), 'Thursday', 'Bi-Weekly', 'Large', NULL);
```

ROUTE

```
INSERT INTO ROUTE VALUES (RouteID_value.nextval, 'Polokwane North', 'Polokwane Suburbs', 'David Molefe', '09:00');
INSERT INTO ROUTE VALUES (RouteID_value.nextval, 'Tzaneen Central', 'Tzaneen Town', 'Mary Phiri', '10:30');
```

DELIVERY

```
INSERT INTO DELIVERY VALUES (DeliveryID_value.nextval, 1, 1, 3, TO_DATE('2025-04-21 08:45:00', 'YYYY-MM-DD HH24:MI:SS'), 'Delivered on time');
INSERT INTO DELIVERY VALUES (DeliveryID_value.nextval, 2, 2, 1, NULL, 'Scheduled for next week');
```

PAYMENT

```
INSERT INTO PAYMENT VALUES (PaymentID_value.nextval, 1, 1, NULL, 200.00, TO_DATE('2025-04-01', 'YYYY-MM-DD'), 'TXN12345');
INSERT INTO PAYMENT VALUES (PaymentID_value.nextval, NULL, 2, 1, 1500.00, TO_DATE('2025-04-20', 'YYYY-MM-DD'), 'TXN67890');
```

BOXCONTENT

```
INSERT INTO BOXCONTENT VALUES (BoxContentID_value.nextval, 1, 1, 2);
INSERT INTO BOXCONTENT VALUES (BoxContentID_value.nextval, 1, 2, 1);
```

2.4. Creating Views

Views create virtual tables for efficient querying, reducing complexity for Royal Fields staff. We create two views to support common reports.

2.4.1. Member Subscription View

This view shows delivery details, joining DELIVERY, MEMBER, ROUTE, and DELIVERYSTATUSLOOKUP.

```sql
CREATE OR REPLACE VIEW MemberSubscriptionView AS
SELECT
    m.MEMBER_ID,
    m.FirstName || ' ' || m.LastName AS MemberName,
    s.SUBSCRIPTION_ID,
    s.DeliveryFrequency,
    s.BoxSize,
    st.SubscriptionTypeName
FROM MEMBER m
JOIN SUBSCRIPTION s ON m.MEMBER_ID = s.MEMBER_ID
JOIN SUBSCRIPTIONTYPELOOKUP st ON m.SUBSCRIPTION_TYPE_ID = st.SUBSCRIPTION_TYPE_ID;
```

2.4.2. Delivery Status View

This view shows delivery details, joining DELIVERY, MEMBER, ROUTE, and DELIVERYSTATUSLOOKUP.

```sql
CREATE OR REPLACE VIEW DeliveryStatusView AS
SELECT
    d.DELIVERY_ID,
    m.FirstName || ' ' || m.LastName AS MemberName,
    r.RouteName,
    ds.DeliveryStatusName,
    d.ActualDeliveryTime
FROM DELIVERY d
JOIN MEMBER m ON d.MEMBER_ID = m.MEMBER_ID
JOIN ROUTE r ON d.ROUTE_ID = r.ROUTE_ID
JOIN DELIVERYSTATUSLOOKUP ds ON d.DELIVERY_STATUS_ID = ds.DELIVERY_STATUS_ID;
```

2.5. Creating Indexes

Indexes improve query performance on frequently accessed columns. Primary keys automatically have indexes, so we add indexes on foreign keys and search fields.

2.5.1. SUBSCRIPTION Table Indexes

The idx_subscription_member index is created on the MEMBER_ID column of the SUBSCRIPTION table, which is a foreign key referencing the MEMBER table. This index speeds up queries that retrieve subscription details for specific members, such as identifying a member's delivery schedule or preferences. For example, Royal Fields staff frequently query subscriptions to plan weekly or bi-weekly deliveries, and this index reduces the time needed to join the SUBSCRIPTION and MEMBER tables. By optimizing

lookups on MEMBER_ID, the index enhances the efficiency of subscription management in the CSA system.

```
CREATE INDEX idx_subscription_member ON SUBSCRIPTION(MEMBER_ID);
```

2.5.2. DELIVERY Table Indexes

The idx_delivery_member index is created on the MEMBER_ID column of the DELIVERY table, a foreign key referencing the MEMBER table. This index improves the performance of queries that track deliveries for specific members, such as checking delivery statuses or delivery dates for customers in Limpopo. Royal Fields staff rely on these queries to monitor delivery progress and respond to customer inquiries, and the index ensures faster data retrieval when joining the DELIVERY and MEMBER tables. This optimization supports timely and accurate delivery tracking in the CSA system.

```
CREATE INDEX idx_delivery_member ON DELIVERY(MEMBER_ID);
```

2.5.3. INVENTORY Table Indexes

The idx_inventory_produce index is created on the PRODUCE_ID column of the INVENTORY table, a foreign key referencing the PRODUCE table. This index accelerates queries that retrieve inventory details for specific produce items, such as checking the availability of culturally relevant crops like Amaranth (Morogo) or Mango. Royal Fields staff use these queries to plan deliveries and manage stock levels, and the index reduces the time required to join the INVENTORY and PRODUCE tables. By improving query performance, this index supports efficient inventory management in the CSA system.

```
CREATE INDEX idx_inventory_produce ON INVENTORY(PRODUCE_ID);
```

3.SQL Select Statements

3.1. Description of Usage

The following table maps the marking guide criteria to the SQL statements provided in Section 3.2, ensuring all required SQL features are demonstrated.

Criterion	**Location (in Section 3.2)**
Limitation of rows	Statements 1, 3, 5, 7, 11
Limitation of columns	Statements 1–3, 5–7, 11–12, 15–16
Sorting	Statements 1, 3, 5, 7, 11–12, 15–16
LIKE	Statements 3, 11
AND	Statements 1, 5–7, 11
OR	Statements 3, 7
Variables	Statements 3, 7, 11
Character functions	Statement 12 (CONCAT, SUBSTR)
Round	Statements 1, 9, 14
Date functions	Statement 10 (SYSDATE, MONTHS_BETWEEN)
Aggregate functions	Statements 4, 8–10, 13–14, 19
Group by	Statements 4, 8–10, 13–14
Having	Statements 9–10
Join	Statements 1, 6–7, 9, 11–14, 16
Sub-queries	Statements 8–10, 13–14, 16
Extra Functionality	Statements 2 (CASE), 4 (CASE), 17 (UPDATE), 18 (DELETE)

3.2. SQL SELECT Statements and Their Uses

Statement 1

Displays member names, subscription types, and rounded subscription prices, limited to 5 rows. Helps staff manage subscriptions.

```
SELECT
    m.FirstName || ' ' || m.LastName AS MemberName,
    st.SubscriptionTypeName,
    ROUND(st.Price, 2) AS SubscriptionPrice
FROM MEMBER m
JOIN SUBSCRIPTIONTYPELOOKUP st ON m.SUBSCRIPTION_TYPE_ID = st.SUBSCRIPTION_TYPE_ID
WHERE ROWNUM <= 5
ORDER BY m.LastName;
```

Statement 2

Shows produce availability, using CASE to indicate low stock. Helps inventory management.

```
SELECT
    p.ProduceName,
    i.QuantityAvailable,
    CASE
        WHEN i.QuantityAvailable = 0 THEN 'Out of Stock'
        WHEN i.QuantityAvailable < 20 THEN 'Low Stock'
        ELSE 'In Stock'
    END AS StockStatus
FROM INVENTORY i
JOIN PRODUCE p ON i.PRODUCE_ID = p.PRODUCE_ID
WHERE i.QuantityAvailable IN (0, 10, 30, 50);
```

Statement 3

Finds members by name, using LIKE and variables. Assists in member lookup.

```
SELECT *
FROM MEMBER
WHERE FirstName LIKE '%' || &name || '%'
   OR LastName LIKE '%' || &name || '%'
ORDER BY FirstName ASC;
```

Statement 4

Counts deliveries by route, using CASE for active routes. Helps optimize delivery logistics.

```
SELECT
    r.RouteName,
    COUNT(d.DELIVERY_ID) AS DeliveryCount,
    CASE
        WHEN COUNT(d.DELIVERY_ID) > 0 THEN 'Active'
        ELSE 'Inactive'
    END AS RouteStatus
FROM ROUTE r
LEFT JOIN DELIVERY d ON r.ROUTE_ID = d.ROUTE_ID
GROUP BY r.RouteName
ORDER BY DeliveryCount DESC;
```

Statement 5

Lists members with weekly subscriptions, limiting columns. Supports subscription planning.

```
SELECT
    m.FirstName,
    m.LastName,
    s.DeliveryDay
FROM MEMBER m
JOIN SUBSCRIPTION s ON m.MEMBER_ID = s.MEMBER_ID
WHERE s.DeliveryFrequency = 'Weekly'
ORDER BY m.LastName;
```

Statement 6

Show farmers and their produce, for inventory tracking.

```
SELECT
    f.FarmerName,
    p.ProduceName,
    i.QuantityAvailable
FROM FARMER f
JOIN INVENTORY i ON f.FARMER_ID = i.FARMER_ID
JOIN PRODUCE p ON i.PRODUCE_ID = p.PRODUCE_ID
WHERE i.QuantityAvailable > 0;
```

Statement 7

Displays delivery details for a specific status, using variables and OR. Tracks delivery progress.

```
SELECT
    d.DELIVERY_ID,
    m.FirstName || ' ' || m.LastName AS MemberName,
    ds.DeliveryStatusName
FROM DELIVERY d
JOIN MEMBER m ON d.MEMBER_ID = m.MEMBER_ID
JOIN DELIVERYSTATUSLOOKUP ds ON d.DELIVERY_STATUS_ID = ds.DELIVERY_STATUS_ID
WHERE ds.DeliveryStatusName = '&status1' OR ds.DeliveryStatusName = '&status2'
ORDER BY d.DELIVERY_ID;
```

Statement 8

Counts payments by type, using aggregate functions. Supports financial reporting.

```
SELECT
    pt.PaymentTypeName,
    COUNT(p.PAYMENT_ID) AS PaymentCount
FROM PAYMENT p
JOIN PAYMENTTYPELOOKUP pt ON p.PAYMENT_TYPE_ID = pt.PAYMENT_TYPE_ID
GROUP BY pt.PaymentTypeName;
```

Statement 9

Shows average payment amounts by type, with HAVING clause. Identifies high-value payment methods.

```
SELECT
    pt.PaymentTypeName,
    ROUND(AVG(p.Amount), 2) AS AvgPayment
FROM PAYMENT p
JOIN PAYMENTTYPELOOKUP pt ON p.PAYMENT_TYPE_ID = pt.PAYMENT_TYPE_ID
GROUP BY pt.PaymentTypeName
HAVING AVG(p.Amount) > &min_amount;
```

Statement 10

Calculates subscription duration using date functions and subqueries. Helps track active subscriptions.

```
SELECT
    s.SUBSCRIPTION_ID,
    m.FirstName,
    m.LastName,
    TRUNC(MONTHS_BETWEEN(SYSDATE, s.StartDate), 2) AS MonthsActive
FROM SUBSCRIPTION s
JOIN MEMBER m ON s.MEMBER_ID = m.MEMBER_ID
WHERE s.MEMBER_ID IN (
    SELECT MEMBER_ID
    FROM SUBSCRIPTION
    WHERE EndDate > SYSDATE
);
```

Statement 11

Finds deliveries by member name, using LIKE and joins. Assists in delivery tracking.

```
SELECT
    m.FirstName,
    m.LastName,
    d.DELIVERY_ID,
    ds.DeliveryStatusName
FROM MEMBER m
JOIN DELIVERY d ON m.MEMBER_ID = d.MEMBER_ID
JOIN DELIVERYSTATUSLOOKUP ds ON d.DELIVERY_STATUS_ID = ds.DELIVERY_STATUS_ID
WHERE (m.FirstName LIKE '%' || &member_name || '%'
   OR m.LastName LIKE '%' || &member_name || '%')
   AND ROWNUM <= 5;
```

Statement 12

Generates a delivery code using character functions, for secure delivery tracking.

```
SELECT
    d.DELIVERY_ID,
    RPAD(
        CONCAT(
            SUBSTR(m.FirstName, 1, 2),
            d.DELIVERY_ID
        ),
        8, '*'
    ) AS DeliveryCode
FROM DELIVERY d
JOIN MEMBER m ON d.MEMBER_ID = m.MEMBER_ID
ORDER BY d.DELIVERY_ID;
```

Statement 13

Counts produce delivered, using subqueries. Tracks produce distribution.

```
SELECT
    (SELECT ProduceName FROM PRODUCE WHERE PRODUCE_ID = bc.PRODUCE_ID) AS ProduceName,
    SUM(bc.Quantity) AS TotalDelivered
FROM BOXCONTENT bc
GROUP BY bc.PRODUCE_ID
ORDER BY TotalDelivered DESC;
```

Statement 14

Shows average produce quantities per delivery, using ROUND. Monitors box contents.

```
SELECT
    (SELECT ProduceName FROM PRODUCE WHERE PRODUCE_ID = bc.PRODUCE_ID) AS ProduceName,
    ROUND(AVG(bc.Quantity), 2) AS AvgQuantity
FROM BOXCONTENT bc
GROUP BY bc.PRODUCE_ID
ORDER BY AvgQuantity DESC;
```

Statement 15

Lists recent payments, for financial oversight.

```
SELECT
    p.PAYMENT_ID,
    p.Amount,
    p.PaymentDate
FROM PAYMENT p
ORDER BY p.PaymentDate DESC;
```

Statement 16

Calculates subscription duration in weeks, using date functions and joins. Identifies long-term members.

```
SELECT
    m.FirstName || ' ' || m.LastName AS MemberName,
    TRUNC(MONTHS_BETWEEN(SYSDATE, s.StartDate) * 4, 2) AS WeeksActive
FROM SUBSCRIPTION s
JOIN MEMBER m ON s.MEMBER_ID = m.MEMBER_ID
ORDER BY WeeksActive DESC;
```

Statement 17

Updates delivery status, demonstrating extra functionality.

```
UPDATE DELIVERY
SET DELIVERY_STATUS_ID = &new_status_id
WHERE DELIVERY_ID = &delivery_id;
```

Statement 18

Deletes paid farmer payments, demonstrating extra functionality.

```
DELETE FROM PAYMENT
WHERE FARMER_ID = &farmer_id AND Amount <= &max_amount;
```

Statement 19

Counts active subscriptions using the MemberSubscriptionView, for subscription management

```
CREATE OR REPLACE VIEW MemberSubscriptionView AS
SELECT
    m.MEMBER_ID,
    m.FirstName || ' ' || m.LastName AS MemberName,
    s.SUBSCRIPTION_ID,
    s.DeliveryFrequency,
    s.BoxSize,
    st.SubscriptionTypeName
FROM MEMBER m
JOIN SUBSCRIPTION s ON m.MEMBER_ID = s.MEMBER_ID
JOIN SUBSCRIPTIONTYPELOOKUP st ON m.SUBSCRIPTION_TYPE_ID = st.SUBSCRIPTION_TYPE_ID;
```

Statement 20

Counts deliveries per member, helping Royal Fields track member engagement.

```
SELECT
    m.FirstName || ' ' || m.LastName AS MemberName,
    COUNT(d.DELIVERY_ID) AS DeliveryCount
FROM MEMBER m
LEFT JOIN DELIVERY d ON m.MEMBER_ID = d.MEMBER_ID
GROUP BY m.MEMBER_ID, m.FirstName, m.LastName
ORDER BY DeliveryCount DESC;
```